

4.10.9

Anshu Kumar Ram - EE25BTECH11009

October 2025

Question

Find the equation of the plane passing through the line of intersection of the planes

$$\mathbf{r} \cdot (\hat{i} + 2\hat{j} + 3\hat{k}) - 4 = 0, \quad (1)$$

$$\mathbf{r} \cdot (2\hat{i} + \hat{j} - \hat{k}) + 5 = 0 \quad (2)$$

and which is perpendicular to the plane

$$\mathbf{r} \cdot (5\hat{i} + 3\hat{j} - 6\hat{k}) + 8 = 0. \quad (3)$$

Solution - Step 1

The required plane can be written as a linear combination of the first two planes

$$(\mathbf{n}_1 + \lambda \mathbf{n}_2)^T \mathbf{r} = c_1 + \lambda c_2 \quad (4)$$

where

$$\mathbf{n}_1 = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}, \quad c_1 = 4 \quad (5)$$

$$\mathbf{n}_2 = \begin{pmatrix} 2 \\ 1 \\ -1 \end{pmatrix}, \quad c_2 = -5 \quad (6)$$

and it is perpendicular to

$$\mathbf{r} \cdot \begin{pmatrix} 5 \\ 3 \\ -6 \end{pmatrix} + 8 = 0 \quad (7)$$

Solution - Step 2

For perpendicular planes,

$$\mathbf{n}^T \mathbf{n}_3 = 0, \quad \mathbf{n}_3 = \begin{pmatrix} 5 \\ 3 \\ -6 \end{pmatrix} \quad (8)$$

Substituting,

$$(\mathbf{n}_1 + \lambda \mathbf{n}_2)^T \mathbf{n}_3 = 0 \quad (9)$$

$$\begin{pmatrix} 1 & 2 & 3 \end{pmatrix} \begin{pmatrix} 5 \\ 3 \\ -6 \end{pmatrix} + \lambda \begin{pmatrix} 2 & 1 & -1 \end{pmatrix} \begin{pmatrix} 5 \\ 3 \\ -6 \end{pmatrix} = 0 \quad (10)$$

$$-7 + 19\lambda = 0 \implies \lambda = \frac{7}{19} \quad (11)$$

Solution - Final Step

$$\mathbf{n} = \mathbf{n}_1 + \frac{7}{19}\mathbf{n}_2 = \frac{1}{19} \begin{pmatrix} 33 \\ 45 \\ 50 \end{pmatrix} \quad (12)$$

$$c = c_1 + \lambda c_2 = 4 + \frac{7}{19}(-5) = \frac{41}{19} \quad (13)$$

Multiplying throughout by 19:

$$\begin{pmatrix} 33 & 45 & 50 \end{pmatrix} \mathbf{r} = 41 \quad (14)$$

Final Answer:

$$33x + 45y + 50z = 41 \quad (15)$$

C Code: func.c

```
1  #include <stdio.h>
2
3  void find_plane(double *out) {
4      double n1[3] = {1, 2, 3};
5      double n2[3] = {2, 1, -1};
6      double n3[3] = {5, 3, -6};
7      double c1 = -4, c2 = 5;
8
9      double num = 7.0, den = 19.0;
10     double lambda = num / den;
11
12     double n[3];
13     n[0] = n1[0] + lambda * n2[0];
14     n[1] = n1[1] + lambda * n2[1];
15     n[2] = n1[2] + lambda * n2[2];
16
17     double d = c1 + lambda * c2;
18     out[0] = n[0];
19     out[1] = n[1];
20     out[2] = n[2];
21     out[3] = d;
22 }
```

Python + C Integration (Part 1)

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import ctypes
4
5 lib = ctypes.CDLL('./func.so')
6 out = (ctypes.c_double * 4)()
7 lib.find_plane(out)
8
9 n = np.array([out[0], out[1], out[2]])
10 d = out[3]
11 n_int, d_int = 19 * n, 19 * d
12
13 print(f"Plane: {n_int[0]:.0f}x + {n_int[1]:.0f}y + \
14 {n_int[2]:.0f}z + {d_int:.0f} = 0")
```

Python + C Integration (Part 2)

```
1 x, y = np.linspace(-5,5,20), np.linspace(-5,5,20)
2 X, Y = np.meshgrid(x, y)
3 Z = (-d_int - n_int[0]*X - n_int[1]*Y) / n_int[2]
4
5 fig = plt.figure()
6 ax = fig.add_subplot(111, projection='3d')
7 ax.plot_surface(X, Y, Z, alpha=0.6, color='skyblue')
8 Z3 = (-8 - 5*X - 3*Y)/(-6)
9 ax.plot_surface(X, Y, Z3, alpha=0.3, color='orange')
10 plt.savefig("../figs/Figure_2.png")
11 plt.show()
```


Pure Python Implementation (Part 1)

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from mpl_toolkits.mplot3d import Axes3D
4 from matplotlib.patches import Patch
5 from matplotlib.lines import Line2D
6
7 # Step 1: Define the given data
8 n1 = np.array([1, 2, 3])
9 n2 = np.array([2, 1, -1])
10 n3 = np.array([5, 3, -6])
11 c1, c2, c3 = 4, -5, -8
12
13 lambda_val = -np.dot(n1, n3) / np.dot(n2, n3)
14 n = n1 + lambda_val * n2
15 c = c1 + lambda_val * c2
16 n, c = 19*n, 19*c
```

Pure Python Implementation (Part 2)

```
1 x = np.linspace(-5, 5, 30)
2 y = np.linspace(-5, 5, 30)
3 X, Y = np.meshgrid(x, y)
4 Z1 = (c1 - n1[0]*X - n1[1]*Y) / n1[2]
5 Z2 = (c2 - n2[0]*X - n2[1]*Y) / n2[2]
6 Z = (c - n[0]*X - n[1]*Y) / n[2]
7
8 fig = plt.figure(figsize=(9,7))
9 ax = fig.add_subplot(111, projection='3d')
10 ax.plot_surface(X, Y, Z1, color='cyan', alpha=0.5)
11 ax.plot_surface(X, Y, Z2, color='orange', alpha=0.5)
12 ax.plot_surface(X, Y, Z, color='green', alpha=0.6)
```

Pure Python Implementation (Part 3)

```
1 d = np.cross(n1, n2)
2 A = np.vstack((n1, n2, d))
3 b = np.array([c1, c2, 0])
4 r0 = np.linalg.solve(A, b)
5
6 t = np.linspace(-3, 3, 50)
7 line_points = r0[:, None] + d[:, None]*t
8 ax.plot(line_points[0], line_points[1],
9         line_points[2], color='k', linewidth=3)
10
11 ax.set_xlabel('X')
12 ax.set_ylabel('Y')
13 ax.set_zlabel('Z')
14 ax.set_title("Plane Intersection and Required Plane")
15 ax.view_init(elev=20, azimuth=40)
16 plt.show()
```

Plot

